

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
За седьмой семестр
По дисциплине: «Модели решения задач в интеллектуальных системах»
Тема: Сравнение функций потерь MSE и BCE для многослойного
персептрона в задаче XOR

Выполнил:
Студент 4 курса
Группы ИИ-26(2)
Семёнов А.В.
Проверил:
Никитин С.К.

Брест 2026

Сравнение MSE и BCE в задаче XOR

1 Условие работы и вариант задания

Цель работы — на основе персептрона 2–2–1 из лабораторной работы №1 реализовать обучение с бинарной кросс-энтропией и сравнить его с MSE по скорости, устойчивости и точности восстановления числовых классов. Во всех вычислительных нейронах используется сигмоида; веса обновляются после каждого примера.

Вариант 1: $c_0 = 0$, $c_1 = -6$. Входные значения и целевые классы задаются полной таблицей XOR. Программа использует только NumPy и Matplotlib; прямой проход и обратное распространение ошибки реализованы вручную.

А	В	Цель y	Метка t
0	0	0	0
0	-6	-6	1
-6	0	-6	1
-6	-6	0	0

Согласование с первой лабораторной

В методичке конфигурация А одновременно названа базовой из ЛР №1 и описана как обучение сигмоиды непосредственно на исходных c_0/c_1 . Для варианта 1 второе условие несовместимо с диапазоном сигмоиды (0; 1): значение -6 недостижимо. В ЛР №1 уже применялась нормализация и обратное преобразование выхода.

Основная конфигурация А сохраняет фактическую реализацию ЛР №1: MSE в нормированной шкале с обратным пересчётом. Конфигурация Б отличается от неё функцией потерь BCE и соответствующим выходным градиентом. Дополнительно проведены пять запусков A_raw — буквального варианта с целями 0 и -6 и непосредственным сигмоидным выходом. Его результаты приведены отдельно и не смешиваются с корректным сравнением А и Б.

Нормализация и восстановление шкалы

$$x_1 = -A/6; \quad x_2 = -B/6; \quad t = (y - c_0)/(c_1 - c_0) = -y/6$$
$$y_{real} = c_0 + (c_1 - c_0)p = -6p; \quad p = \sigma(z)$$

Числовой диапазон восстановленного выхода — (-6; 0). Класс выбирается по минимальному расстоянию до 0 и -6; при равенстве выбран 0. Граница классов соответствует $p = 0,5$ и $y_{real} = -3$. Преобразование применяется без округления и без обрезания входов, в том числе при вводе значений из $[-10; 10]$.

2 Архитектура и алгоритм обучения

Архитектура не изменилась: два входа, два скрытых нейрона и один выходной нейрон; всего девять обучаемых параметров. Обозначим входные веса W_{ij} , смещения скрытого слоя b_j , выходные веса v_j и выходное смещение b_o .

$$h_j = \sigma(\sum_i x_i W_{ij} + b_j); \quad z = \sum_j h_j v_j + b_o; \quad p = \sigma(z) \\ \sigma(z) = 1/(1 + \exp(-z)); \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Функции потерь и выходные градиенты

Для совместимости с ЛР №1 под суммарной ошибкой MSE понимается половина суммы квадратов. Это явно выбранный масштаб квадратичной потери; обычное среднее квадратов для четырёх примеров равно $Es_MSE/2$.

$$L_MSE = \frac{1}{2}(p - t)^2; \quad Es_MSE = \sum_k L_MSE_k \\ L_BCE = -t \ln(p) - (1 - t) \ln(1 - p); \quad Es_BCE = \sum_k L_BCE_k \\ \partial L_BCE / \partial p = (p - t) / (p(1 - p)); \quad \partial p / \partial z = p(1 - p) \\ \delta_{o,BCE} = \partial L_BCE / \partial z = p - t; \quad \delta_{o,MSE} = (p - t)p(1 - p)$$

В BCE множитель $p(1 - p)$ сокращается на выходном слое. Поэтому неверный уверенный прогноз не подавляет выходной градиент дополнительной производной сигмоиды. На скрытом слое производная сигмоиды сохраняется:

$$\delta_j = \delta_o v_j h_j (1 - h_j); \quad \partial L / \partial W_{ij} = x_i \delta_j; \quad \partial L / \partial b_j = \delta_j \\ \partial L / \partial v_j = h_j \delta_o; \quad \partial L / \partial b_o = \delta_o; \quad \theta \leftarrow \theta - \eta \partial L / \partial \theta$$

Все градиенты вычисляются до изменения весов. BCE рассчитывается устойчиво через логит: $\max(z, 0) - tz + \ln(1 + \exp(-|z|))$. После каждой эпохи суммарная ошибка заново вычисляется на всех четырёх примерах при единых текущих весах.

Параметр	Значение
Скорость обучения η	0,5
Порог E_e для каждой функции	0,01
Предел эпох	50 000
Seed	42, 43, 44, 45, 46
Инициализация	NumPy default_rng; $U[-0,5; 0,5)$
Порядок обучения	Фиксированный, как в таблице XOR

При одном seed все конфигурации получают одинаковые начальные веса. Перезапуски и подбор удачного seed не выполняются. Для графиков и режима функционирования заранее выбран первый seed = 42. Остановка: $Es \leq E_e$ или исчерпание предела эпох.

3 Результаты серии экспериментов

Результаты получены запуском lab2_variant1.py, NumPy 2.3.5. Accuracy рассчитана по ближайшим числовым классам; $MAE = \frac{1}{4}\sum|y_{real} - y|$. Все итоговые показатели относятся к последним весам соответствующего запуска.

Сеть	Seed	Эпохи	Es	Accuracy	MAE	Es ≤ Ee
A MSE	42	3069	0.009990953	100%	0.418804	Да
A MSE	43	2707	0.009993059	100%	0.418918	Да
A MSE	44	1919	0.009993810	100%	0.419551	Да
A MSE	45	2981	0.009989098	100%	0.418942	Да
A MSE	46	1910	0.009986908	100%	0.422936	Да
Б BCE	42	2331	0.009996476	100%	0.014975	Да
Б BCE	43	2418	0.009998959	100%	0.014979	Да
Б BCE	44	2279	0.009996110	100%	0.014975	Да
Б BCE	45	2439	0.009997042	100%	0.014976	Да
Б BCE	46	50000	1.390927398	75%	1.500221	Нет

Конфигурация А достигла порога в 5 из 5 запусков: диапазон 1910–3069 эпох, среднее 2517,2. Конфигурация Б достигла порога в 4 из 5 запусков: среди успешных 2279–2439 эпох, среднее 2366,75. Среднее число выполненных эпох BCE с учётом неуспешного запуска — 11 893,4. Пять запусков характеризуют эту серию, а не вероятность сходимости в общем случае.

При seed = 42 BCE достигла порога на 738 эпох раньше MSE. При seed = 44, напротив, MSE остановилась раньше (1919 против 2279). Первое достижение accuracy = 100% при seed = 42 зарегистрировано на эпохах 2262 для MSE и 389 для BCE. Это общий классификационный показатель, но он не является критерием остановки и не означает устойчивого сохранения точности на всех следующих эпохах.

Буквальная конфигурация из методички

A_raw seed	Эпохи	Es	Accuracy	MAE
42	50000	36.000013647	50%	3.000001
43	50000	36.000013517	50%	3.000001
44	50000	36.000013468	50%	3.000001
45	50000	36.000013594	50%	3.000001
46	50000	36.000013756	50%	3.000001

Ни один запуск A_raw не сошёлся. При непосредственном выходе $p > 0$ ошибка на каждой цели –6 больше 36, поэтому $Es_{raw} = \frac{1}{2}\sum(p - y)^2 > 36$; $MAE > 3$. Выход всегда ближе к 0, чем к –6, и accuracy равна 50%. Это ограничение диапазона выходной функции, а не доказательство непригодности MSE.

4 Скорость и устойчивость сходимости

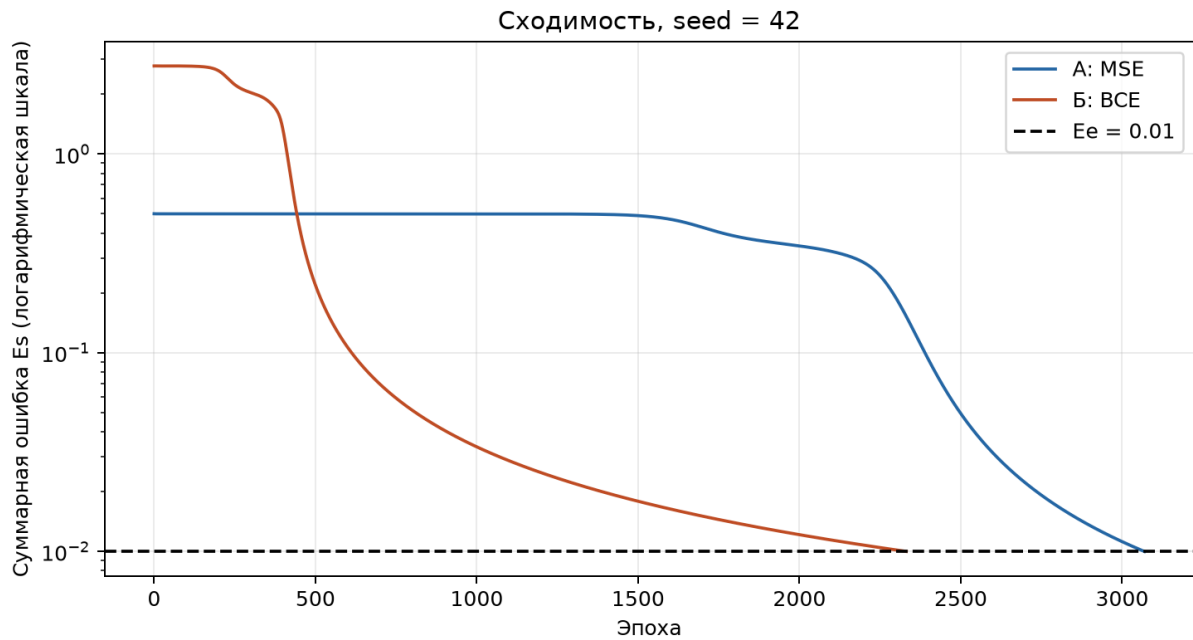


Рисунок 1 — Суммарная ошибка при seed 42

На рисунке 1 показаны собственные функции потерь двух конфигураций; шкала E_s логарифмическая. Равенство $E_e = 0,01$ задаёт разные требования к точности. Численные высоты кривых MSE и BCE нельзя сравнивать как одну и ту же меру качества.

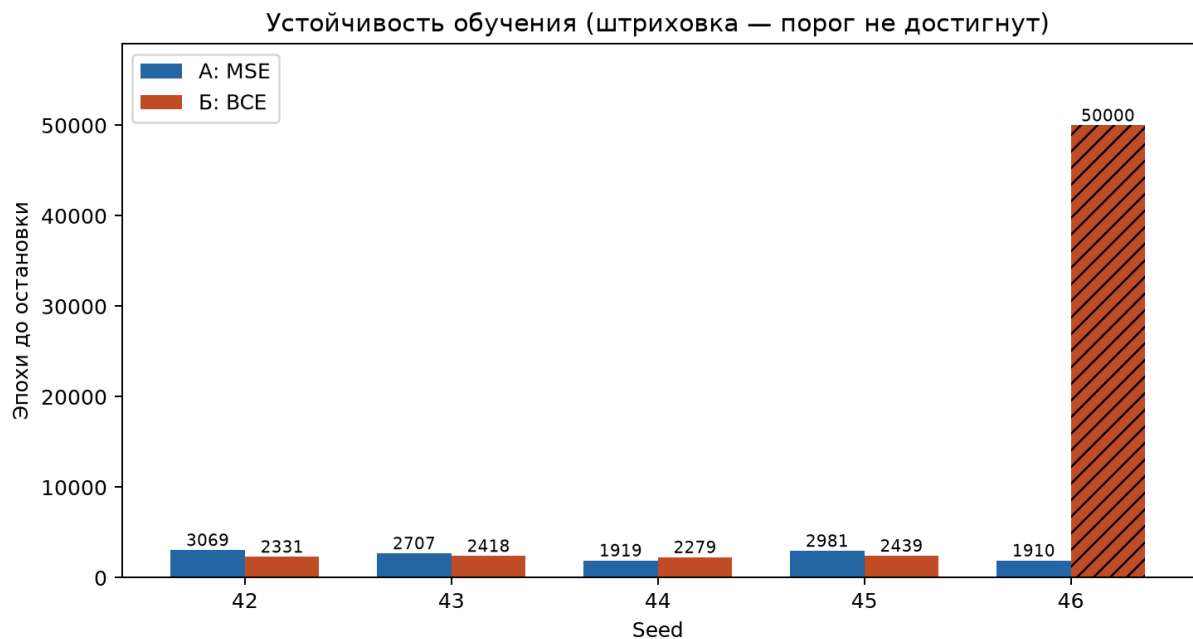


Рисунок 2 — Число эпох в пяти запусках

Штриховкой отмечена остановка BCE при seed = 46 по пределу 50 000 эпох. Итоговые выходы $p \approx (0,000047; 0,999946; 0,466698; 0,466743)$: два последних примера почти не различаются. Это неудачное обучение с плато ошибки; по одной траектории нельзя строго установить попадание в локальный минимум.

5 Разделяющая поверхность и восстановление шкалы

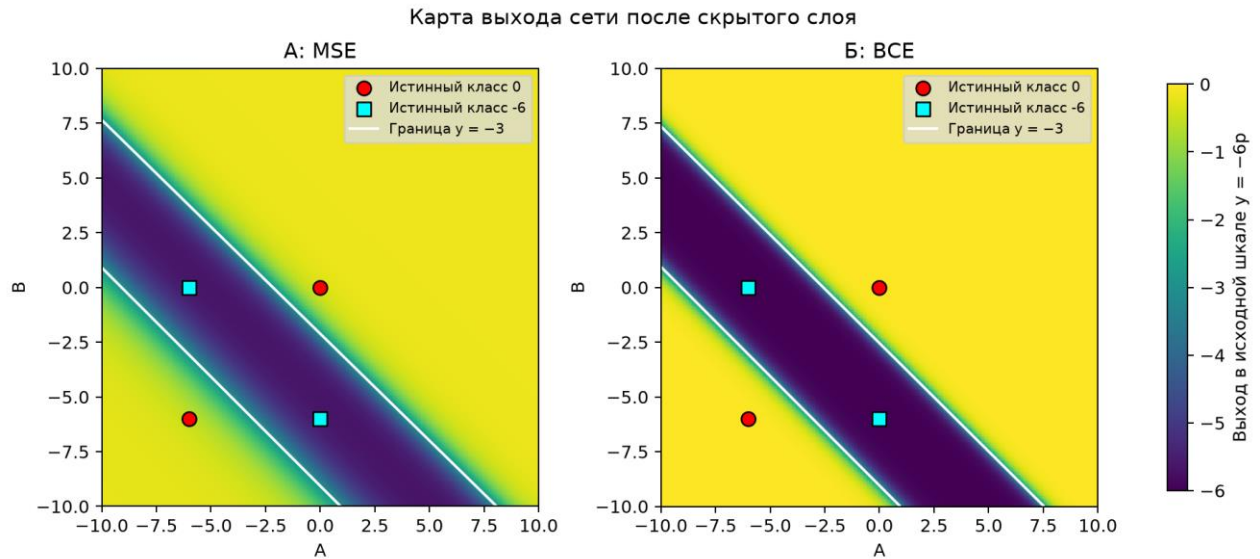


Рисунок 3 — Карты выхода А и Б на диапазоне $[-10; 10]^2$

Для обеих карт выбрана общая исходная шкала от -6 до 0 : она позволяет непосредственно видеть близость к числовым классам. Точки показывают истинные метки обучающей выборки; белые линии — границу $y_{\text{real}} = -3$. Изображена результирующая карта сети после преобразования скрытым слоем.

В обоих случаях класс -6 занимает диагональную полосу, содержащую $(0; -6)$ и $(-6; 0)$; точки $(0; 0)$ и $(-6; -6)$ находятся снаружи. Такая область соответствует мягкому пересечению двух полуплоскостей, создаваемых скрытыми нейронами. BCE даёт более резкие переходы и более насыщенные выходы. Точная форма вдали от четырёх обучающих точек не задана таблицей XOR.

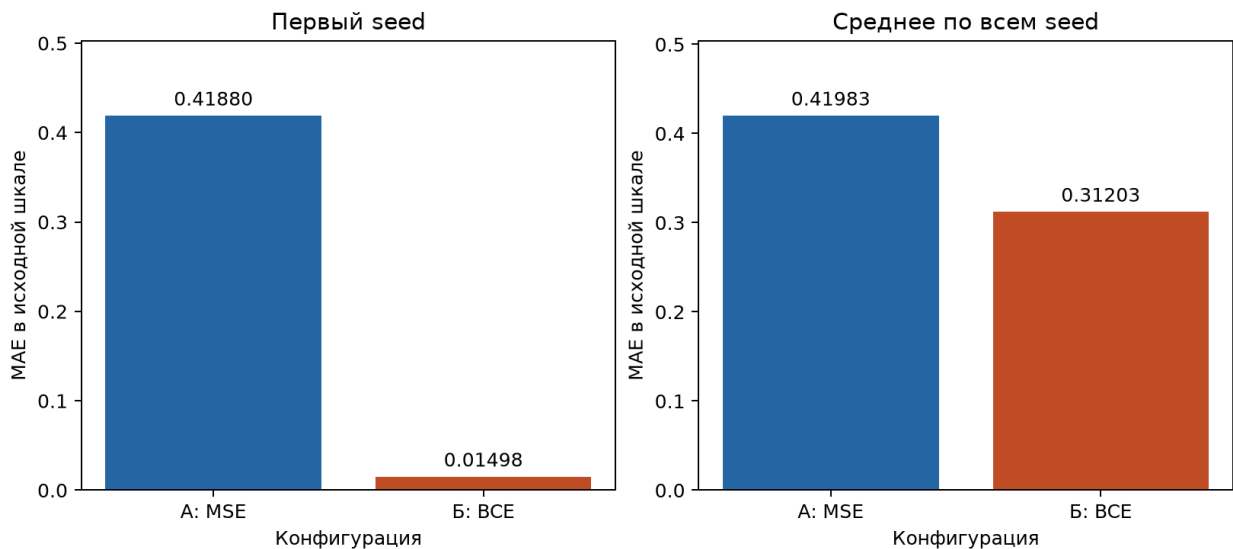


Рисунок 4 — Средняя абсолютная ошибка в исходной шкале

При $\text{seed} = 42$ MAE снизилась с $0,418804$ у MSE до $0,014975$ у BCE. Средняя MAE по всем пяти запускам равна $0,419830$ и $0,312025$ соответственно; у BCE учтён неуспешный запуск с $\text{MAE} = 1,500221$. Эти результаты зависят от порогов остановки и не доказывают общего превосходства BCE в регрессии.

6 Режим функционирования и проверка программы

После обучения выводятся восемь проверочных пар, затем можно вводить А и В из диапазона $[-10; 10]$. Для каждого входа программа показывает нормированный выход с четырьмя знаками, пересчитанный выход и ближайший класс. Ниже приведены результаты первого $\text{seed} = 42$.

A; B	p MSE	y MSE	Класс А	p BCE	y BCE	Класс Б
0; 0	0.0572	-0.343109	0	0.0021	-0.012707	0
0; -6	0.9332	-5.599342	-6	0.9978	-5.986596	-6
-6; 0	0.9324	-5.594695	-6	0.9977	-5.986471	-6
-6; -6	0.0877	-0.526143	0	0.0034	-0.020261	0
-1; -5	0.9331	-5.598649	-6	0.9978	-5.986576	-6
-5; -1	0.9326	-5.595553	-6	0.9977	-5.986493	-6
-1; -1	0.4401	-2.640549	0	0.0950	-0.569702	0
-5; -5	0.2817	-1.690096	0	0.0557	-0.334198	0

Первые четыре строки — обучающие примеры, следующие четыре — новые пары. Для новых пар эталон XOR не задан: они демонстрируют интерполяцию сети и не включены в ассигасу. За пределами интервала $[-6; 0]$ по каждой координате выполняется экстраполяция, хотя весь разрешённый диапазон ввода остаётся $[-10; 10]$.

Проверка корректности

Градиенты всех девяти параметров проверены центральными конечными разностями с шагом 10^{-6} для MSE, BCE с метками 0 и 1 и A_gaw. Максимальное абсолютное расхождение — $2,908 \cdot 10^{-9}$ при допуске 10^{-7} . Для BCE проверена конечность потерь на логитах ± 1000 .

Проверены воспроизводимость повторных коротких запусков, обратимость нормализации классов, выбор класса 0 при равенстве расстояний, границы допустимого ввода и отклонение NaN, бесконечности и значений вне диапазона. Во всех успешных экспериментах ошибка на предпоследней эпохе выше порога, а на последней — не выше. MSE при $\text{seed} = 42$ точно воспроизвела 3069 эпох и $E_s = 0,009990953272218741$ из исходного кода ЛР №1.

Запуск программы

```
python -m pip install numpy matplotlib
python lab2_variant1.py
python lab2_variant1.py --self-test
python lab2_variant1.py --no-input --no-show
```

Результаты JSON и четыре PNG сохраняются в папку lab2_results рядом со скриптом. --out позволяет выбрать другую папку; --seeds задаёт не менее пяти разных инициализаций, --lr, --ee и --max-epochs — параметры обучения. При обычном запуске после закрытия окон графиков доступен ввод пар; q завершает работу. Десятичная запятая поддерживается.

7 Выводы

Скорость и устойчивость

ВСЕ быстрее достигла 100% классификационной точности в четырёх успешных парных запусках, что согласуется с отсутствием дополнительного множителя $p(1 - p)$ в выходном градиенте. Однако при выбранных η и seed MSE оказалась устойчивее по критерию остановки: 5/5 против 4/5. Следовательно, преимущество ВСЕ в форме выходного градиента не гарантирует глобальную сходимость всей нелинейной сети.

Точность числовых выходов

В успешном запуске seed = 42 ВСЕ точнее восстановила исходные числа: MAE = 0,014975 против 0,418804. Обязательный компромисс между классификацией и восстановлением шкалы здесь не обнаружен. Но одинаковое $E_e = 0,01$ не уравнивает точность: из $E_s_MSE \leq 0,01$ следует лишь $MAE_real \leq 6\sqrt{(2 \cdot 0,01/4)} \approx 0,424264$; для ВСЕ неравенство $-\ln(1 - e) \geq e$, где $e = |p - t|$, даёт $MAE_real \leq 6 \cdot 0,01/4 = 0,015$. Наблюдаемый разрыв во многом объясняется разной строгостью порогов. Для сравнения при одинаковой регрессионной точности нужно отдельно задавать общий порог MAE.

Случайная инициализация

Разброс MSE составил 1159 эпох; среди успешных ВСЕ — 160 эпох. Меньший разброс успешной части ВСЕ нельзя трактовать как большую общую надёжность: seed = 46 завершился с accuracy = 75% и $E_s = 1,390927398$. Перезапуска этого опыта не было; плато совместимо с неудачной конфигурацией скрытых признаков, но само по себе не доказывает строгий локальный минимум.

Линейная неразделимость и архитектура

Кодирование 0 и -6 связано с бинарным кодированием обратимым аффинным преобразованием, поэтому сохраняет линейную неразделимость XOR. При пороге сигмоиды один нейрон задаёт прямую и не может отделить противоположные вершины квадрата. Замена MSE на ВСЕ меняет оптимизацию, но не класс разделяющих границ однослойной модели. Скрытый слой по-прежнему необходим для сети с двумя исходными признаками; двух скрытых нейронов достаточно для полученных решений.

Геометрия решения

Карты обеих обученных конфигураций при seed = 42 согласуются с XOR на четырёх заданных точках: два класса разделены двумя переходными границами, образующими диагональную полосу. ВСЕ формирует более резкое изменение выхода. Полоса является допустимым продолжением таблицы на плоскость, но таблица из четырёх точек не задаёт единственную правильную границу на всём диапазоне входов. Неудача A_raw отдельно показывает необходимость согласовать область значений активации с целевыми числами.

Приложение А Полный код программы

```
import argparse
import json
from pathlib import Path

import numpy as np

C0, C1 = 0., -6.
RAW_X = np.array([[0., 0.], [0., -6.], [-6., 0.], [-6., -6.]])
RAW_Y = np.array([0., -6., -6., 0.])
CHECK_X = np.vstack((RAW_X, [[-1., -5.], [-5., -1.], [-1., -1.], [-5., -5.])))
NAMES = {'MSE': 'A: MSE', 'BCE': 'Б: BCE', 'RAW': 'A_raw: MSE на 0/-6'}
```



```
def normalize(v):
    return (np.asarray(v, dtype=float) - C0) / (C1 - C0)
```



```
def decode(v):
    return C0 + (C1 - C0) * np.asarray(v)
```



```
def sigmoid(v):
    v = np.asarray(v, dtype=float)
    e = np.exp(-np.abs(v))
    return np.where(v >= 0, 1. / (1. + e), e / (1. + e))
```



```
def nearest(v):
    v = np.asarray(v)
    return np.where(np.abs(v - C0) <= np.abs(v - C1), C0, C1)
```



```
class MLP:
    def __init__(self, seed):
        rng = np.random.default_rng(seed)
        self.w1 = rng.uniform(-.5, .5, (2, 2))
        self.b1 = rng.uniform(-.5, .5, 2)
        self.w2 = rng.uniform(-.5, .5, 2)
        self.b2 = np.array(rng.uniform(-.5, .5))

    def forward(self, x):
        h = sigmoid(np.asarray(x) @ self.w1 + self.b1)
        z = h @ self.w2 + self.b2
        return h, z, sigmoid(z)

    def gradients(self, x, target, loss):
        h, _, p = self.forward(x)
        # Отличие BCE: производная сигмоиды сокращается с dL/dp.
        delta = p - target
        if loss != 'BCE':
            delta *= p * (1. - p)
        # Все градиенты вычисляются ДО обновления выходных весов.
        hidden_delta = delta * self.w2 * h * (1. - h)
        return (np.outer(x, hidden_delta), hidden_delta,
                delta * h, np.asarray(delta))

    def parameters(self):
        return self.w1, self.b1, self.w2, self.b2

    def step(self, x, target, loss, lr):
        grads = self.gradients(x, target, loss)
        for param, grad in zip(self.parameters(), grads):
            param -= lr * grad
```



```
def loss_sum(model, x, targets, loss):
    _, z, p = model.forward(x)
    if loss == 'BCE':
        # BCE из логитов: без log(0), clipping и переполнения exp.
        return float(np.sum(np.maximum(z, 0) - targets * z
```

```

        + np.log1p(np.exp(-np.abs(z))))))
return float(.5 * np.sum((p - targets) ** 2))

def metrics(model, loss):
    targets = RAW_Y if loss == 'RAW' else normalize(RAW_Y)
    p = model.forward(normalize(RAW_X))[2]
    real = p if loss == 'RAW' else decode(p)
    return {'Es': loss_sum(model, normalize(RAW_X), targets, loss),
            'accuracy': float(np.mean(nearest(real) == RAW_Y)),
            'mae': float(np.mean(np.abs(real - RAW_Y))),
            'p': p.tolist(), 'real': real.tolist()}

def train(loss, seed, lr, ee, max_epochs):
    model = MLP(seed)
    x = normalize(RAW_X)
    targets = RAW_Y if loss == 'RAW' else normalize(RAW_Y)
    history = [loss_sum(model, x, targets, loss)]
    # Общие диагностические критерии не управляют остановкой.
    first_acc = 0 if metrics(model, loss)['accuracy'] == 1 else None
    first_mae = 0 if metrics(model, loss)['mae'] <= .1 else None
    for epoch in range(1, max_epochs + 1):
        if history[-1] <= ee:
            break
        for row, target in zip(x, targets):
            model.step(row, target, loss, lr)
        m = metrics(model, loss)
        history.append(m['Es'])
        if first_acc is None and m['accuracy'] == 1:
            first_acc = epoch
        if first_mae is None and m['mae'] <= .1:
            first_mae = epoch
    m = metrics(model, loss)
    m.update(seed=seed, epochs=len(history)-1, converged=m['Es'] <= ee,
             history=history, first_accuracy_100=first_acc,
             first_mae_le_01=first_mae,
             weights=[a.tolist() for a in model.parameters()])
    return model, m

def predict(model, a, b):
    pair = np.asarray([a, b], dtype=float)
    if not np.all(np.isfinite(pair)) or np.any(np.abs(pair) > 10):
        raise ValueError('A и B должны быть конечными числами из [-10; 10].')
    p = float(model.forward(normalize(pair))[2])
    real = float(decode(p))
    return {'A': float(a), 'B': float(b), 'p': p,
            'real': real, 'class': float(nearest(real))}

def visualize(results, models, ee, out, show):
    import matplotlib
    if not show:
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 10})
    colors = {'MSE': '#2466a4', 'BCE': '#bf4c24'}
    def save(fig, name):
        fig.savefig(out / name, dpi=180, bbox_inches='tight')

    fig, ax = plt.subplots(figsize=(8, 4.3), layout='constrained')
    for loss in ('MSE', 'BCE'):
        h = results[loss][0]['history']
        ax.semilogy(np.arange(len(h)), h, label=NAMES[loss], color=colors[loss])
        ax.axhline(ee, color='black', ls='--', label=f'Ee = {ee:g}')
        ax.set(xlabel='Эпоха', ylabel='Суммарная ошибка Es (логарифмическая шкала)',
              title=f'Сходимость, seed = {results["MSE"][0]["seed"]}')
        ax.grid(alpha=.25); ax.legend(); save(fig, 'convergence.png')

    fig, ax = plt.subplots(figsize=(8, 4.3), layout='constrained')
    pos = np.arange(len(results['MSE']))
    for k, loss in enumerate(('MSE', 'BCE')):
```

```

runs = results[loss]
bars = ax.bar(pos + (k-.5)*.36, [r['epochs'] for r in runs],
              .36, label=NAMES[loss], color=colors[loss])
for bar, r in zip(bars, runs):
    if not r['converged']:
        bar.set_hatch('///')
    ax.annotate(str(r['epochs']), (bar.get_x()+bar.get_width()/2,
                                   bar.get_height()), ha='center', va='bottom', fontsize=8)
ax.set_xticks(pos, [str(r['seed']) for r in results['MSE']])
ax.set(xlabel='Seed', ylabel='Эпохи до остановки',
       title='Устойчивость обучения (штриховка – порог не достигнут)')
ax.margins(y=.18); ax.legend(); save(fig, 'epochs.png')

grid = np.linspace(-10, 10, 241)
aa, bb = np.meshgrid(grid, grid)
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5), layout='constrained')
for ax, loss in zip(axes, ('MSE', 'BCE')):
    model = models[loss]
    values = decode(model.forward(normalize(np.c_[aa.ravel(), bb.ravel()])))
    values = values.reshape(aa.shape)
    im = ax.pcolormesh(aa, bb, values, cmap='viridis', vmin=-6, vmax=0,
                       shading='auto')
    if values.min() < -3 < values.max():
        ax.contour(aa, bb, values, levels=[-3], colors='white',
                   linewidths=1.5, linestyle='solid')
    for cls, marker, color in [(0, 'o', 'red'), (-6, 's', 'cyan')]:
        pts = RAW_X[RAW_Y == cls]
        ax.scatter(pts[:, 0], pts[:, 1], marker=marker, c=color, s=65,
                   edgecolor='black', label=f'Истинный класс {cls}')
    ax.plot([], [], color='white', label='Граница y = -3')
    ax.set(xlabel='A', ylabel='B', title=NAMES[loss], xlim=(-10, 10), ylim=(-10, 10))
    ax.set_aspect('equal'); ax.legend(fontsize=8, facecolor='#dddddd')
fig.colorbar(im, ax=axes, label='Выход в исходной шкале y = -6p', shrink=.85)
fig.suptitle('Карта выхода сети после скрытого слоя')
save(fig, 'surfaces.png')

fig, axes = plt.subplots(1, 2, figsize=(9, 4), layout='constrained')
for ax, mean in zip(axes, (False, True)):
    vals = [np.mean([r['mae'] for r in results[l]]) if mean
            else results[l][0]['mae'] for l in ('MSE', 'BCE')]
    bars = ax.bar(['A: MSE', 'B: BCE'], vals, color=list(colors.values()))
    ax.bar_label(bars, fmt='%.5f', padding=3)
    ax.set(ylabel='MAE в исходной шкале', xlabel='Конфигурация',
          title='Среднее по всем seed' if mean else 'Первый seed')
    ax.margins(y=.2)
save(fig, 'mae.png')
if show:
    plt.show()
plt.close('all')

def self_test():
    max_error = 0.
    x = np.array([.2, .8])
    for loss, target in (('MSE', 1.), ('BCE', 1.), ('BCE', 0.), ('RAW', -6.)):
        model = MLP(42)
        grads = model.gradients(x, target, loss)
        for param, grad in zip(model.parameters(), grads):
            for idx in np.ndindex(param.shape):
                old = float(param[idx]); eps = 1e-6
                param[idx] = old + eps
                plus = loss_sum(model, x, target, loss)
                param[idx] = old - eps
                minus = loss_sum(model, x, target, loss)
                param[idx] = old
                max_error = max(max_error, abs((plus-minus)/(2*eps)-grad[idx]))
    assert max_error < 1e-7, max_error
    assert np.array_equal(decode(normalize(RAW_Y)), RAW_Y)
    assert float(nearest(-3.)) == C0
    model = MLP(42)
    for bad in [(float('nan'), 0), (0, float('inf')), (-10.01, 0)]:
        try:
            predict(model, *bad)

```

```

except ValueError:
    pass
else:
    raise AssertionError('Недопустимый ввод принят')
predict(model, -10, 10)
for loss in ('MSE', 'BCE', 'RAW'):
    _, a = train(loss, 42, .5, .01, 10)
    _, b = train(loss, 42, .5, .01, 10)
    assert a == b
    assert a['epochs'] == 10 and len(a['history']) == 11
    assert a['Es'] == a['history'][-1]
# Устойчивость вычисления BCE при больших логитах.
for z in (-1000., 1000.):
    model.w2[:] = 0; model.b2[...] = z
    for target in (0., 1.):
        assert np.isfinite(loss_sum(model, x, target, 'BCE'))
print(f'Проверки пройдены; максимальная ошибка градиента {max_error:.3e}')

def main():
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)
    parser.add_argument('--seeds', type=int, nargs='+', default=[42, 43, 44, 45, 46])
    parser.add_argument('--lr', type=float, default=.5)
    parser.add_argument('--ee', type=float, default=.01)
    parser.add_argument('--max-epochs', type=int, default=50000)
    parser.add_argument('--out', type=Path,
                        default=Path(__file__).resolve().parent / 'lab2_results')
    parser.add_argument('--no-input', action='store_true')
    parser.add_argument('--no-show', action='store_true')
    parser.add_argument('--self-test', action='store_true')
    args = parser.parse_args()
    if args.self_test:
        self_test(); return
    if (not np.isfinite(args.lr) or args.lr <= 0 or not np.isfinite(args.ee)
        or args.ee <= 0 or args.max_epochs < 1 or min(args.seeds) < 0
        or len(set(args.seeds)) != len(args.seeds) or len(args.seeds) < 5):
        parser.error('Нужны >=5 разных seed >=0, конечные lr>0, ee>0, max-epochs>=1.')
    args.out.mkdir(parents=True, exist_ok=True)
    results, models = {}, {}
    for loss in ('MSE', 'BCE', 'RAW'):
        results[loss] = []
        for seed in args.seeds:
            model, r = train(loss, seed, args.lr, args.ee, args.max_epochs)
            results[loss].append(r)
            if seed == args.seeds[0]:
                models[loss] = model
        print(f'{NAMES[loss]:24} seed={seed} эпох={r["epochs"]:5d} '
              f'Es={r["Es"]:.9f} acc={r["accuracy"]:.0%} '
              f'MAE={r["mae"]:.6f} критерий={r["converged"]}', flush=True)
    checks = {loss: [predict(models[loss], *pair) for pair in CHECK_X]
              for loss in ('MSE', 'BCE')}
    data = {'parameters': {'seeds': args.seeds, 'lr': args.lr, 'ee': args.ee,
                          'max_epochs': args.max_epochs,
                          'numpy_version': np.__version__, 'results': results, 'checks': checks}
           (args.out / 'results.json').write_text(
               json.dumps(data, ensure_ascii=False, indent=2, encoding='utf-8'))
    for loss, rows in checks.items():
        print('\n' + NAMES[loss])
        for r in rows:
            print(f'A={r["A"]:.6f}, B={r["B"]:.6f}: p={r["p"]:.4f}, '
                  f'y={r["real"]:.6f}, класс={r["class"]:.6f}')
    visualize(results, models, args.ee, args.out, not args.no_show)
    print(f'Результаты и графики: {args.out.resolve()}')
    if args.no_input:
        return
    print('Введите A B из [-10; 10], q – выход. Для новых пар эталон не задан.')
    while True:
        try:
            line = input('A B > ').strip()
        except (EOFError, KeyboardInterrupt):
            print(); break
        if line.lower() in ('q', 'quit', 'exit', 'выход'):

```

```

        break
    try:
        vals = list(map(float, line.replace(',', '.').split()))
        if len(vals) != 2:
            raise ValueError('Введите ровно два числа через пробел.')
        for loss in ('MSE', 'BCE'):
            r = predict(models[loss], *vals)
            print(f'{NAMES[loss]}: p={r["p"]:.4f}, y={r["real"]:.6f}, '
                  f'ближайший класс={r["class"]:g}')
    except ValueError as exc:
        print(f'Ошибка ввода: {exc}')

if __name__ == '__main__':
    main()

```